

Множествено наследяване - клас може да наследи функционалност от повече от един базов клас.

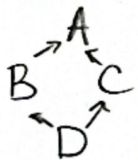
⊕ повете повторно използване на код

⊕ реализира интерфейс + поведение

⊖ конфликти на имена

⊖ диамантен проблем

Диамантен проблем - повторно присъствие на една и съща базова част



• D ще има два отделни A подобекта

• Конструкторът на A ще се извика 2 пъти

Решение: Virtual наследяване - гарантира един единствен shared базов подобект

• Конструкторите на B и C не конструират A

Object slicing - при копиране на наследен обект по стойност в базов тип, се губи частта на наследения

Решение: - използване на указатели
- използване на референции
- контейнери с умни указатели

Клониране на полиморфни обекти

Имаме полиморфна йерархия и искаме:

• да копираме обекти ~~не~~ през базов клас

• да запазим реалния (динамичен) тип

• да избегнем object slicing

Решение: clone() → virtual функция (=0)

• Базовия клас я декларира

• Производните класове я презаписват и връщат копие от собствения си тип.

Колекция от полиморфни обекти

• Контейнер, съхраняващ обекти от различни конкретни типове, но ги третира чрез общ базов интерфейс.

Добри практики:

- Композиция през множествено наследяване
- Множествено наследяване за интерфейси е безопасно
- При диамантен проблем → virtual наследяване
- Избягвайте slicing - съхранявайте полиморфни обекти чрез smart pointers
- За копиране — виртуален clone()
- базовия деструктор е virtual